L11-v9

2026年10月25日 16:51

子代理（Subagent）的核心原理是上下文隔离。主Agent（父代理）可以将探索、调研这类“过程繁琐、结论精简”的子任务，委托给一个拥有独立上下文窗口的子代理去完成。子代理在完成任务后，只向主Agent返回一份精炼的摘要，从而避免原始过程的中间内容（如大量文件内容、搜索结果）污染主对话的上下文。这种模式能够有效保持主上下文的整洁，通过限制工具来强制约束子代理行为，并可将特定任务路由到更便宜、更快的模型上以控制成本。

基本原理：上下文隔离与结果回收

子代理机制建立在两个核心原则之上：

- 上下文隔离不变量：子代理的记忆（M_child）并非父代理记忆（M_parent）的切片，两者不共享存储。子代理从干净的状态启动，看不到父代理之前的对话历史，父代理也仅能看到子代理的最终结果，而非其中间过程。这从根本上避免了共享内存的并发写入风险。
- 结果回收契约：隔离的代价是，父子间的通信必须通过一个显式的工具调用接口来完成，而非隐式共享内存。因此，在派发任务时就必须定义好输出契约（如要求摘要包含“绝对路径”），并据此校验回传结果，确保父代理能正确解析。

主流技术方案

业界在实现子代理时，侧重点有所不同，形成了几种典型模式：

- Claude Code：多模式与深度限制：支持进程内、Fork子进程、Git Worktree和远程容器四种派生模式，通过分层控制实现从轻量到强隔离的灵活性。通过 maxDepth 参数硬性限制递归深度（默认通常为3），防止无限递归。子代理默认只读，Write/Edit等工具被拒，并可通过 toolFilter 精确控制可见工具集。
- AutoGen：对话驱动的协作：将多智能体协作建模为“群聊”，智能体通过结构化消息传递进行协商与迭代。提供 RoundRobinGroupChat（固定顺序）、SelectorGroupChat（LLM动态选择）等多种群聊模式，适应不同协作场景。
- DeepSeek Harness：声明式控制：通过 persona（角色）、toolFilter（工具过滤）和 maxDepth（深度限制）三个独立维度来精确控制子代理的启动配置。
- 成本追踪与可观测性：由于子代理独立计费，需要专门的追踪机制。Claude Code的子代理请求计入同一使用限制；而 agentprof、pyagent-trace 等工具可将会话呈现为调用树，按节点和子树汇总token消耗与成本。

当前版本的实现方式

用 readOnlyView() 和 kTaskToolName 两个关键设计解决了安全与递归问题。

- readOnlyView() 与 K-40 问题：Task 工具本身的 risk 也是 Read，如果仅按风险等级过滤，子代理将能再次派生子代理，导致递归深度失控。解决方案是在 readOnlyView() 中按名字显式排除 Task 工具，从源头切断递归能力。同时，该视图会复制工具的完整闭包（std::function），确保子代理拿到的工具是真正可执行的，而不仅仅是定义。
- kTaskToolName 常量：将工具名定义为 tool_registry.hpp 中的常量，是为了避免 subagent.hpp 与 tool_registry.hpp 之间的循环依赖，并确保注册与过滤时使用的名字完全一致，防止拼写漂移。
- 四个隔离边界的落地：在 SubagentRunner::run() 中，从四个维度落实隔离：① 使用全新的 ContextAssembler 和独立的 system 提示，避免复制主上下文；② 通过 readOnlyView() 获得安全工具集；③ 使用 NoUI（confirm 恒为 false）确保后台运行无弹窗；④ 将子代理的 Usage 单独记账并汇总到主会话，避免成本统计失真。
  子代理机制是Agent架构中解决“上下文污染”和“探索成本”问题的关键设计，其核心在于通过严格的隔离与契约，将昂贵的探索过程封装在独立的、可控的沙箱中执行。

新增清单

文件	变化类型	核心新增
subagent.hpp	新增文件	SubagentSpec / SubagentResult / SubagentRunner / kSubagentSummaryPrompt / makeTaskToolDef()
subagent.cpp	新增文件	SubagentRunner::run()——四个隔离边界的落地
tool_registry.hpp	修改	① 新增 kTaskToolName 常量；② 新增 readOnlyView() 方法
tool_registry.cpp	修改	实现 readOnlyView()：按 risk 过滤 + 按名字排除 Task + 闭包一起复制

main.cpp	重大修改	① provider 创建位置提前；② 注册 Task 工具；③ 子代理成本记账；④ 自检

subagent 模块

子代理获得自己的全新上下文，完全与主对话隔离。它的工作不会让主上下文膨胀。完成后只返回摘要。
子代理有自己的状态（SubagentSpec / Result）、自己的策略（工具子集、步数上限、可选模型）

四个隔离边界

上下文

```
// ★ 不复用主会话的 system：主 system 里有用户需求/项目规则/仓库地图，
//   把它塞给子代理等于**把主上下文又复制了一份**，隔离就白做了。
ContextAssembler assembler(BudgetConfig{}, nullptr);
assembler.setSystem(std::string(kSubagentSummaryPrompt));
```

工具

```
// ★ 用 readOnlyView()：闭包一起复制过来，工具是真的能执行的。
ToolRegistry subRegistry = registry_.readOnlyView();
```

UI

```
// NoUI 是 **fail-closed**：confirm() 恒 false。
// 子代理在后台跑，没有人会看到它的权限弹窗 —
// "没有 UI 时的正确行为是拒绝，不是允许"（L4 定的规矩，本讲第一次真正用上）。
NoUI ui;
```

成本

```
// ★ token 不进主上下文，但**钱是花了**。
//   子代理的用量记在它自己的 loop 上，主循环的 spentTokens() 一个数都看不到
//   → 漏掉这条，`/cost` 报出来的数字就是**假的**（K-42）。
out.usage = loop.totalUsage();
```

```
constexpr std::string_view kSubagentSummaryPrompt = R"(
你是完成一个子任务的编码助手。完成后输出一份结构化摘要，必须包含：
...
3. 涉及文件的**绝对路径**
...
)";
```

```
ToolRegistry ToolRegistry::readOnlyView() const {
    ToolRegistry v;
    // 权限门与沙箱要**一起带过去**：否则子代理的工具没有权限判定，
    // 而"没有门"在权限系统里等于"全放行"（fail-open 是最坏的默认）。
    v.gate_    = gate_;
    v.sandbox_ = sandbox_;
    for (const auto& e : entries_) {
        if (e.def.risk != RiskLevel::Read) continue;
        // ★★ K-40：不给递归能力。Task 的 risk 也是 Read，按属性过滤拦不住它。
        if (e.def.name == kTaskToolName) continue;
        v.entries_.push_back(e);        // ★ 闭包一起复制 — 工具要能真的执行
    }
    return v;
}
```

嵌套环

```
LoopLimits limits;
limits.maxSteps       = spec.maxSteps;
limits.maxTokensSpent = spec.maxTokens;

AgentLoop loop(provider_, subRegistry, assembler, limits);

Turn turn;
turn.input = spec.goal;          // ★ 全新的空 history

LoopOutcome outcome = loop.run(turn, hooks);
```

```
if (outcome == LoopOutcome::Completed) {
    out.summary = text;
} else {
    out.failed = true;
    out.error  = std::string(toString(outcome));
    out.summary = "[子代理未完成: " + out.error + "]\n已获得的部分信息:\n" + text;
}
```

Task 工具的执行体

```
const std::string goal = args.value("goal", std::string());
if (goal.empty()) return ToolResult::error("", "goal 不能为空");

af::SubagentSpec spec;
spec.goal     = goal;
spec.readOnly = args.value("read_only", true);
spec.maxSteps = std::max(1, envInt("AF_SUBAGENT_STEPS", 30));

af::SubagentRunner runner(*pp, registry);
const af::SubagentResult r = runner.run(spec);

// ★ 记账：token 不进主上下文，但钱要记到主会话
subagentUsage->inputTokens  += r.usage.inputTokens;
subagentUsage->outputTokens += r.usage.outputTokens;
++(*subagentRuns);

std::cout << "[子代理] 目标=" << goal.substr(0, 40)
          << "  步数=" << runner.lastSteps()
          << "  读了 " << r.filesRead.size() << " 个文件"
          << "  摘要 " << r.summary.size() << " 字符"
          << "  token " << (r.usage.inputTokens + r.usage.outputTokens)
          << (r.failed ? "  ★未完成" : "")
          << (r.truncated ? "  ★摘要被截断" : "")
          << "\n" << std::flush;

return ToolResult::ok("", r.summary);
```

完整数据流

```
主模型看到 Task 工具定义
  │
  ▼ 决定"这个探索任务值得派个子代理"
  │
  ▼ 调用 Task({goal: "这个仓库里认证怎么处理？"})
  │
  ▼ main.cpp 的 Task lambda
  │
  ├─ SubagentRunner runner(*pp, registry)
  └─ runner.run(spec)
      │
      ├─ 边界❷: registry.readOnlyView()
      │    ├─ 只留 risk==Read
      │    ├─ 排除 Task (K-40)
      │    └─ 闭包一起复制
      │
      ├─ 边界❷: ContextAssembler(BudgetConfig{}, nullptr)
      │    └─ setSystem(kSubagentSummaryPrompt)  ← 全新的 system
      │
      ├─ 边界❷: NoUI (fail-closed)
      │
      ├─ 环里套环: AgentLoop loop(provider_, subRegistry, assembler, limits)
      │    └─ turn.input = spec.goal  ← 全新的空 history
      │
      ├─ 循环跑 30 步，读 20 个文件
      │
      └─ 边界❷: out.usage = loop.totalUsage()
  │
  ▼ 返回 SubagentResult{summary, usage, filesRead, ...}
  │
  ▼ main.cpp 记账: subagentUsage += r.usage
  │
  ▼ return ToolResult::ok("", r.summary)
  │
  ▼ 主模型只看到 500 字摘要（20 个文件的内容全在子代理的上下文里，没进来）
```